YOLO, U-Net, IoU & mAP

Sattam Altwaim
satttam.tuuaim@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology



KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 4, Day 26, Self-study

14 October 2026

Table of Contents

1. Where This Fits
2. Learning Outcomes
3. YOLO in Depth
4. U-Net in Depth
5. IoU, Dice, AP and mAP by Hand
6. Olympiad Practice
7. Check Yourself
8. Limitations and What Comes Next
9. Summary
10. References

From the Lecture

- **In the lecture (Day 24):** a box is four numbers in one of three spellings; IoU with numbers; precision, recall and average precision in words, with a measured curve; YOLO's grid on a photo, its $7 \times 7 \times 30$ output, its loss in words, NMS measured, the versions since 2016 and the three ultralytics calls; masks as grids of class numbers, upsampling and the transposed convolution, a small U-Net class, IoU and Dice on a real mask. The labs trained YOLOv8n on road signs and a U-Net on MRI slices.
- **This deck adds:** one YOLO cell decoded into a pixel box and a score; the loss with numbers on that cell; NMS done by hand on four boxes and the case where it deletes a true neighbour; what the modern head predicts (8,400 candidate boxes); how to read `results.csv` and the predict settings; the U-Net's shapes and parameter counts stage by stage, why its input size must divide by 4, and the mask losses with numbers; IoU and Dice on a small grid; a precision-recall curve and its AP by hand, at IoU 0.5 and at 0.75; and the code that computes all of it.
- **How to use it:** every number here was computed and can be redone with pen and paper. Do each worked example yourself before reading its result, then run the code frame.

Why Go Deeper

Where the simple version breaks

- Day 24's AP was a number a library printed. On Friday a leaderboard will print one too, and the only way to know why it moved is to be able to compute it from a list of predictions: six boxes and four objects give $\mathrm{AP} = 0.854$ at IoU 0.5 and 0.375 at IoU 0.75 with the *same* boxes.
- The U-Net class ran on 128×128 slices. Fed a 90×90 image it stops with `Sizes of tensors must match except in dimension 1. Expected size 44 but got size 45`: the floor of Day 25, inside a skip connection.
- A mask model that predicts background everywhere has pixel accuracy 0.75 on our grid and Dice 0. Which loss and which metric you pick decides whether that model looks fine or broken.

After this deck you can

turn a detector's or a segmenter's output into its score by hand, say which setting (confidence, IoU threshold, mask threshold, loss weight) moves that score and in which direction, and read the two models' shapes and logs without guessing.

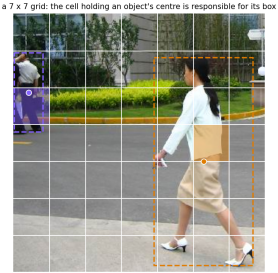
⇒ Learning Outcomes

By the end of this self-study deck, you will be able to:

- **Decode** one YOLO cell into a pixel box and a score, and compute the 2016 loss on that cell term by term.
- **Run** non-maximum suppression by hand on a handful of boxes, and say when it deletes a true object.
- **Read** ultralytics' output: the three scales of the head, the columns of `results.csv`, the confidence and IoU settings of `predict`.
- **Trace** the shapes and parameter counts through a small U-Net, explain the concatenation and the input-size rule, and choose the mask loss with its weight.
- **Compute** IoU, Dice and pixel accuracy on a mask grid, and a precision-recall curve with its AP at two IoU thresholds from a list of predictions.
- **Use** `box_iou`, `nms`, the ultralytics metrics object and a Dice function in code, and say which number a competition reports.

YOLO: **One Cell, Decoded**

Decoding One Cell Into a Box and a Score



Source: Penn-Fudan image FudanPed00005 with the 7×7 grid of Day 24; `scripts/day24_detection_figures.py`.

The cell's numbers (2016 convention)

Cell (row 3, column 5) outputs $x = 0.29$, $y = 0.99$ (centre offset inside the cell, 0 to 1), $w = 0.39$, $h = 0.81$ (size as a fraction of the image), confidence $c = 0.70$, class probabilities $p(\text{person}) = 0.80$.

Cells are 47.9×49.1 pixels on the 335×344 photo. Centre: $c_x = (5 + 0.29) \cdot 47.9 = 253.4$, $c_y = (3 + 0.99) \cdot 49.1 = 195.9$. Size: $W = 0.39 \cdot 335 = 130.7$, $H = 0.81 \cdot 344 = 278.6$. Box: $(253 - 65, 196 - 139, 253 + 65, 196 + 139) = (188, 57, 319, 335)$, Day 24's $(187, 58, 319, 335)$ to the pixel.

Score $= c \cdot p(\text{class}) = 0.70 \cdot 0.80 = 0.56$: how sure the cell is that there is an object, times how sure it is of which one. Every cell produces B such boxes; thresholds and NMS do the rest.

The Loss on That Cell, With Numbers

Redmon et al. (2016): the loss of one responsible cell

$$L = \lambda_{\text{coord}}\left[(x-\hat{x})^2 + (y-\hat{y})^2\right] + \lambda_{\text{coord}}\left[(\sqrt{w}-\sqrt{\hat{w}})^2 + (\sqrt{h}-\sqrt{\hat{h}})^2\right]$$
$$+ (C-\hat{C})^2 + \textstyle\sum_{\text{classes}}(p-\hat{p})^2, \qquad \lambda_{\text{coord}} = 5;\ \text{an empty cell adds } \lambda_{\text{noobj}}\,\hat{C}^2,\ \lambda_{\text{noobj}} = 0.5$$

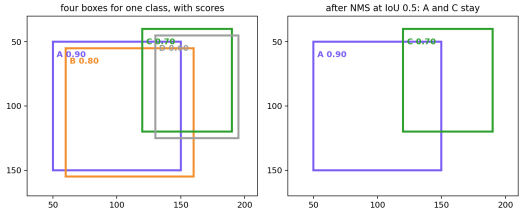
x, y, w, h, p: the truth; C: the IoU of the predicted box with the true one; hats: the cell's predictions.

Truth $(0.29, 0.99, 0.39, 0.81)$, prediction $(0.35, 0.90, 0.30, 0.90)$, $\hat{C} = 0.60$, IoU 0.70

centre $= 5\,[(0.29-0.35)^2+(0.99-0.90)^2] = 5 \cdot 0.0117 = 0.0585$; size $= 5\,[(\sqrt{0.39}-\sqrt{0.30})^2+(\sqrt{0.81}-\sqrt{0.90})^2] = 5\,[0.0059 + 0.0024] = 0.0413$; confidence $= (0.70-0.60)^2 = 0.0100$; class $= (1-0.8)^2 + (0-0.2)^2 = 0.0800$; $L = 0.190$.

- The square roots make the same 0.09 size error cost more on a small box than on this large one; an empty cell predicting $\hat{C} = 0.1$ adds only 0.005, so 47 empty cells cannot drown the two that matter. Ultralytics keeps the three jobs with other formulas: `box_loss`, `cls_loss`, `dfl_loss`.

Non-Maximum Suppression by Hand



Source: four boxes on a 200 × 200 canvas, IoUs from `torchvision.ops.box_iou`, survivors from `torchvision.ops.nms`; `scripts/day26_metrics_figures.py`.

1. Sort by score: A 0.90, B 0.80, C 0.70, D 0.60. Keep A.
2. IoU(A, B): overlap $90 \cdot 95 = 8{,}550$, union $10{,}000 + 10{,}000 - 8{,}550 = 11{,}450$, $0.747 > 0.5$: **delete B**. IoU(A, C) $= 2{,}100/13{,}500 = 0.156$, IoU(A, D) $= 0.109$: both stay for now.
3. Next survivor C: IoU(C, D) $= 4{,}500/6{,}300 = 0.714$: **delete D**.
4. Nothing left: the answer is **A and C**.

When NMS is wrong

Two real signs whose boxes overlap by 0.6 lose one of them at threshold 0.5. Raise it (ultralytics' default at `predict` is `iou=0.7`) when objects touch; lower it when the model draws many loose boxes around one object.

The Modern Head: Three Scales, 8,400 Candidates

YOLOv8 and YOLO11 (ultralytics)

At `imgsz=640` the backbone leaves three maps, with strides 8, 16 and 32: 80×80, 40×40, 20×20 cells, $6{,}400 + 1{,}600 + 400 = 8{,}400$ positions. Each position predicts the **distances to the four box sides** and one score per class; no preset box shapes (anchor-free). Small objects live on the 80×80 map, large ones on the 20×20.

- A 640×640 photo yields $8{,}400 \times (4 + C)$ numbers before thresholds: the 2016 tensor at three scales, one box per position.
- `predict` keeps class scores above `conf` (default 0.25), then NMS at `iou` (default 0.7); `val` uses `conf=0.001` so the curve is complete.

```
r = model.predict("road7.png", conf=0.25,
                  iou=0.7, imgsz=640)[0]
print(r.boxes.xyxy)   # pixels, corners
print(r.boxes.xywhn)  # cx cy w h, 0 to 1
print(r.boxes.conf, r.boxes.cls)
print(r.names[int(r.boxes.cls[0])])
```

`xyxy` for drawing and for submissions in pixels; `xywhn` is Day 24's label spelling. For a task scored by mAP, predict with a **low** `conf` (0.01 to 0.05) and let the metric sweep the thresholds; for a product, pick the `conf` that gives the precision you need.

Reading `results.csv`

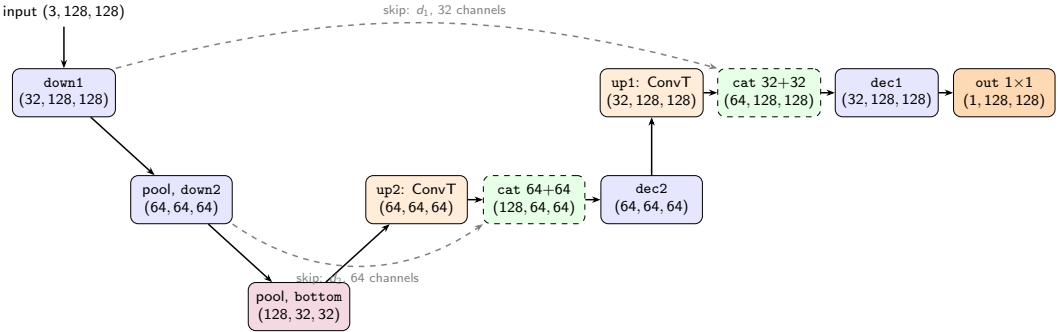
column	what it is	what you want to see
`train/box_loss`	how far the predicted boxes are from the true ones (CIoU, a distance between boxes)	falling steadily
`train/cls_loss`	the class scores against the truth (binary cross-entropy per class)	falling; the first to flatten
`train/dfl_loss`	how sharp the box-side distance predictions are	falling slowly
`val/box_loss`, `val/cls_loss`, `val/dfl_loss`	the same three on the validation split	falling with the training ones; rising means overfitting
`metrics/precision(B)`, `metrics/recall(B)`	at the confidence that maximises F1, on the validation split	both rising; one stuck low tells you which error dominates
`metrics/mAP50(B)`	mean AP over classes at IoU 0.5	the competition number if the rules say mAP@0.5
`metrics/mAP50-95(B)`	mean AP over IoU 0.5 to 0.95 in steps of 0.05	always lower; rewards tight boxes

- **(B)** is for boxes; segmentation models add **(M)** columns. One row per epoch; `best.pt` is the epoch with the best fitness, $0.1 \cdot \text{mAP50} + 0.9 \cdot \text{mAP50-95}$, so "best" is judged by the strict metric. Read it as Day 20 read curves: `val/*` rising while `train/*` fall means stop or augment; mAP still climbing at the last epoch means more epochs are cheap points.

Source: ultralytics documentation (training outputs, `val` and `predict` arguments, the fitness definition). https://docs.ultralytics.com

U-Net: **Shapes, Skips and the Mask Loss**

The Small U-Net, Stage by Stage



`block(c_in, c_out)`: two rounds of Conv 3×3 (padding 1), BatchNorm, ReLU; `ConvT`: `ConvTranspose2d(k=2, s=2)`, halving the channels and doubling the map; `cat` stacks the encoder map of the same size as extra channels. **Parameters:** `down1` 10,272; `down2` 55,680; `bottom` 221,952; `up2` 32,832; `dec2` 110,976; `up1` 8,224; `dec1` 27,840; `out` 33; total 467,809, half of it in `bottom`.

Shapes and counts from Day 24's `UNetSmall` on a (3, 128, 128) input; `scripts/day26_metrics_figures.py`.

The Input-Size Rule, and Why the Original Cropped

Where it breaks

A 90×90 input: $90 \to 45 \to 22$ (floor), up $22 \to 44$, and cat meets d_2 of size 45: `Sizes of tensors must match except in dimension 1. Expected size 44 but got size 45.`

- Two pools need a size divisible by 4, U-Net's four need 16: $100 \to 50 \to 25 \to 50 \to 100$ works. Resize or pad to a multiple of 2^{pools} in the Dataset; resize the mask back.
- The 2015 U-Net had **no padding**: 572×572 in, 388×388 out, skips *cropped* to match (Ronneberger et al., 2015). With `padding=1`, as in ours, sizes match and the output covers the input.

```
net = UNetSmall()
for s in (128, 100, 90):
    x = torch.zeros(1, 3, s, s)
    try:
        print(s, tuple(net(x).shape))
    except RuntimeError as e:
        print(s, "fails:", str(e)[:22])
# 128 (1, 1, 128, 128)
# 100 (1, 1, 100, 100)
# 90 fails: Sizes of tensors must
```

Day 25's test, for a U-Net: zero inputs of every size you might meet, before the Dataset is written.

The Mask Loss, With Numbers

Binary cross-entropy per pixel, with a weight for the rare class

$$\ell = -\big[w_+\, y \log \sigma(z) + (1-y)\log(1-\sigma(z))\big], \qquad \text{Dice loss} = 1 - \frac{2\sum \sigma(z)\, y + 1}{\sum \sigma(z) + \sum y + 1}$$

z: a pixel's logit; $\sigma(z)$: its class probability; $y \in \{0,1\}$: the mask; w_+: `pos_weight`, how many times a missed class pixel costs more than a false alarm.

Four pixels: logits $(2, -2, 0, -4)$ against mask $(1, 0, 1, 0)$

Probabilities $0.881, 0.119, 0.5, 0.018$. BCE per pixel $0.127, 0.127, 0.693, 0.018$, mean 0.241. With $w_+ = 10$ the two class pixels cost 1.269 and 6.931, mean 2.087: the undecided class pixel ($z = 0$) now dominates and gets pushed up. Soft Dice $= (2 \cdot 1.381 + 1)/(1.518 + 2 + 1) = 0.833$, Dice loss 0.167.

- w_+: with 5% tumour pixels the balancing value is $95/5 = 19$; the lab used 10. Too high paints the class everywhere; too low predicts background. Common practice: the **sum** of BCE and Dice loss, one for per-pixel gradients, one for the overlap itself.

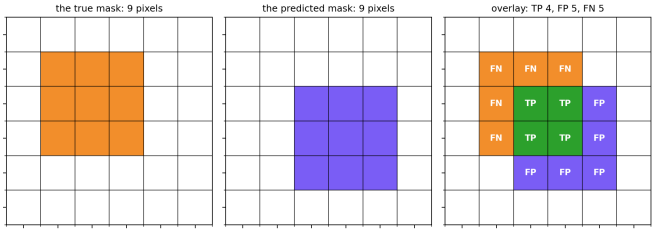
Predicting Masks: Threshold, Resize Back, Save

```
model.eval()
with torch.no_grad():
    for xb, names in test_loader:                                   # xb: (B, 1, 128, 128)
        prob = torch.sigmoid(model(xb.to(device)))                   # 0 to 1 per pixel
        mask = (prob > 0.5).to(torch.uint8)                          # same as logits > 0
        mask = F.interpolate(mask.float(), size=(512, 512), mode="nearest").to(torch.
            uint8)
        for m, n in zip(mask, names):
            Image.fromarray(m[0].cpu().numpy() * 255).save(f"predicted_masks/{n}")
```

- **Threshold** at probability 0.5, which is logit 0. Moving it trades precision for recall like a detector's `conf`: 0.3 paints more, 0.7 less; choose it on the validation Dice, not by eye.
- **Resize back** to the original size with `nearest`, never bilinear: a mask holds class numbers, and a blend of 0 and 1 is not a class.
- **Save** as the task says: a PNG of 0 and 255 per image, or one row per image in a CSV; open one saved mask next to its photo.
- For k classes: k output channels, `CrossEntropyLoss` on a mask of class numbers, `argmax(dim=1)` instead of the threshold.

Metrics: **By Hand, Then in Code**

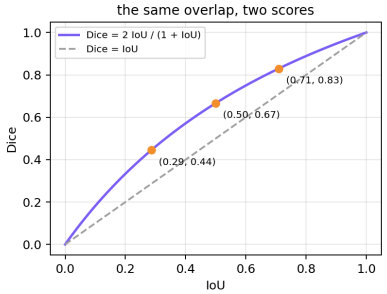
IoU and Dice on a Mask Grid



Source: a 6×6 grid, a 3×3 true mask and a 3×3 predicted mask shifted by one pixel each way; counts from the tensors; `scripts/day26_metrics_figures.py`.

- Count the overlay: $TP = 4$, $FP = 5$, $FN = 5$, $TN = 22$. $\mathrm{IoU} = 4/(4+5+5) = 0.286$; $\mathrm{Dice} = 8/(8+5+5) = 0.444$; pixel accuracy $= 26/36 = 0.722$.
- Predicting **background everywhere**: accuracy $27/36 = 0.750$, better than the model, with $\mathrm{IoU} = \mathrm{Dice} = 0$. Nine pixels off by one in each direction already cost two thirds of the IoU: masks are scored strictly.

Dice Against IoU, and Which One a Task Reports



Source: the identity $\mathrm{Dice} = 2\,\mathrm{IoU}/(1+\mathrm{IoU})$; the dots are the grid above, 0.5, and Day 24's measured mask; `scripts/day26_metrics_figures.py`.

One overlap, two numbers

$\mathrm{Dice} = \dfrac{2\,TP}{2\,TP + FP + FN} = \dfrac{2\,\mathrm{IoU}}{1+\mathrm{IoU}}$: the same ranking of models, a larger number.

$\mathrm{IoU} = 0.286 \rightarrow \mathrm{Dice} = 0.444$; $0.5 \rightarrow 0.667$; Day 24's $0.709 \rightarrow 0.830$.

- **Per image or per dataset?** Dice averaged over images rewards getting every slice roughly right; TP, FP, FN summed over the dataset rewards the large masks. Read the rules; compute what they compute.
- **Mean IoU** over classes (semantic tasks) counts each class once however many pixels it has; **mask mAP** (instance tasks) is detection's mAP with mask IoU in place of box IoU.

A Precision-Recall Curve by Hand

#	score	IoU with its truth	hit at IoU ≥ 0.5?	TP so far	FP so far	precision $= TP/(TP+FP)$	recall $= TP/4$
1	0.95	0.85	yes	1	0	1.00	0.25
2	0.90	0.60	yes	2	0	1.00	0.50
3	0.80	0.30	no	2	1	0.67	0.50
4	0.70	0.80	yes	3	1	0.75	0.75
5	0.60	0.00	no	3	2	0.60	0.75
6	0.50	0.55	yes	4	2	0.67	1.00

One class, 4 true objects, 6 predicted boxes sorted by score; a hit needs IoU ≥ 0.5 with a true box not yet taken. Each hit raises recall by one object (0.25); each miss lowers precision and leaves recall where it was.

- The envelope ignores the dips that a later hit climbs back over: row 3 is forgiven by row 4, row 5 by row 6.

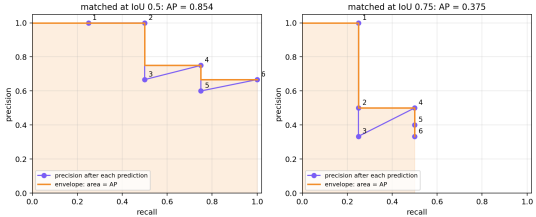
AP, the all-point rule (VOC 2010 onwards)

At each recall r take the **best precision at any recall** $\geq r$, then the area under that staircase:

$$\mathrm{AP} = 0.25\,(1 + 1 + 0.75 + 0.67) = 0.854$$

COCO and ultralytics sample the staircase at 101 recall points instead: 0.856 here. Same idea, a slightly different number; the rules say which.

The Same Boxes at IoU 0.75, and mAP



Source: the six predictions of the table, matched at IoU 0.5 and at 0.75; dots: precision after each prediction; orange: the envelope whose area is AP; `scripts/day26_metrics_figures.py`.

- At IoU 0.75 rows 2 and 6 (IoU 0.60 and 0.55) become misses: recall never passes 0.5 and $\mathrm{AP} = 0.25 \cdot 1 + 0.25 \cdot 0.5 = 0.375$. The boxes did not change; the ruler did.
- **mAP@0.5** averages AP over the classes at this threshold; **mAP@[.5:.95]** first averages over the ten thresholds $0.50, 0.55, \ldots, 0.95$ per class, so a loose-boxed model looks fine on one number and poor on the other. A class with two validation objects has a noisy AP: one miss moves it by half.

In Code: The Metrics Without a Library, and With One

```python
from torchvision import ops
iou = ops.box_iou(pred, truth)   # (P, T)
keep = ops.nms(pred, scores, 0.5)

def ap_from_list(hits, n_true):
    # hits: bools, sorted by score
    tp = hits.float().cumsum(0)
    fp = (~hits).float().cumsum(0)
    prec = tp / (tp + fp)
    rec = tp / n_true
    env = prec.flip(0).cummax(0)[0]
    env = env.flip(0)   # envelope
    r0 = F.pad(rec[:-1], (1, 0))
    return ((rec - r0) * env).sum()
h = torch.tensor([1,1,0,1,0,1]) > 0
print(ap_from_list(h, 4))  # 0.8542
```

```python
m = model.val(data="signs.yaml")
print(m.box.map50, m.box.map)
# mAP@0.5 and mAP@[.5:.95]
print(m.box.ap50)   # AP@0.5 per class
print(m.box.mp, m.box.mr)  # P, R

def dice(logits, y):   # Day 24's lab
    p = (logits > 0).float()
    dims, eps = (1, 2, 3), 1e-6
    inter = (p * y).sum(dims)
    denom = p.sum(dims) + y.sum(dims)
    d = (2 * inter + eps)
    return (d / (denom + eps)).mean()
```

Twelve lines reproduce the table's 0.854; the library adds the matching of boxes to truths (greedy by IoU, each truth once), the per-class loop and the ten thresholds. Use the library for the score, the lines to understand a surprise.

Metrics: **Olympiad Practice**

Practice Task: Reading the Score Before the Leaderboard Does

Task

Kaggle's *Road Sign Detection* (Day 24's Lab 1), scored by mAP@0.5 over the four classes. On the validation split your model's `stop` predictions, sorted by score, have IoUs $0.9, 0.7, 0.4, 0.8, 0.2$ with the five true stop signs' boxes (each matched truth is different), and two `crosswalk` boxes on neighbouring crossings overlap each other with IoU 0.6.

- **Question 1:** compute the `stop` AP@0.5 by hand. If the fifth prediction were removed, would AP change?
- **Question 2:** with `predict(..., iou=0.5)` one of the two crosswalk boxes disappears although both are correct. What happens to recall, and which setting do you change for the submission?
- **Question 3 (segmentation):** on the Brain MRI slices (Lab 2) a model predicts background everywhere: pixel accuracy 0.95, Dice 0. Name the two changes to the loss and the one check on the predicted masks that catch this before the hand-in.

Sources: Kaggle datasets `andrewmvd/road-sign-detection` and `nikhilroxtomar/brain-tumor-segmentation`; the IoU lists are a worked example, not a measured run.

Solution Idea

1. Hits at IoU ≥ 0.5: yes, yes, no, yes, no. Running (TP, FP): $(1,0), (2,0), (2,1), (3,1), (3,2)$; precision $1, 1, 0.67, 0.75, 0.6$; recall $0.2, 0.4, 0.4, 0.6, 0.6$ out of 5 truths. Envelope: 1 up to recall 0.4, then 0.75 up to 0.6, then 0 (two stops were never found).
   $\mathrm{AP} = 0.2 \cdot 1 + 0.2 \cdot 1 + 0.2 \cdot 0.75 = 0.55$. Removing the fifth prediction, a false positive after the last hit, changes nothing: the envelope already ignored it. Extra low-score boxes cannot lower AP, so submit with a low `conf`.
2. NMS keeps the higher-scoring crosswalk and deletes the other because $0.6 > 0.5$: one true object lost, recall for that class drops by one count, and AP with it. Set `iou=0.7` (the default) or higher at `predict`; check the effect on validation mAP, because a higher threshold also keeps more duplicate boxes around single signs.
3. Loss: `BCEWithLogitsLoss(pos_weight=w)` with w near the background-to-tumour pixel ratio (10 to 19 here), and add the Dice loss, which is 1 for an all-background prediction and pushes hard. Check: the fraction of predicted class pixels on the validation set against the true fraction (about 5%); zero means the threshold or the loss is wrong, however good the accuracy looks. Choose the epoch by validation Dice.

Metrics: **Check Yourself**

Check Yourself

Answer before turning the page.

1. **Definition.** A mask has $\mathrm{IoU} = 0.6$. What is its Dice? A second mask has Dice 0.9: its IoU? Which of the two numbers is a competition more likely to report for medical slices, and why does it not matter for ranking?
2. **AP.** Two true objects, four predictions sorted by score with hits yes, no, yes, no. Write the precision and recall after each, the envelope, and the AP. What if the order were yes, yes, no, no?
3. **Code.** Day 24's `UNetSmall` with a third pool and a third `ConvTranspose2d` is fed $(1, 3, 100, 100)$. Does it run? Give the shapes down and up, and the smallest change to the Dataset that makes it run.
4. **YOLO.** You submit predictions made with `conf=0.9`: precision 0.98, recall 0.4 on validation. Your teammate submits the same model with `conf=0.01`. Whose mAP@0.5 is higher, and why can the two files differ by so much while the model is the same?

Answers

1. $\text{Dice} = 2 \cdot 0.6/1.6 = 0.75$. From Dice 0.9: $\text{IoU} = \text{Dice}/(2 - \text{Dice}) = 0.9/1.1 = 0.818$. Medical tasks usually report Dice; the two are a monotone function of each other, so any ranking by one is the same ranking by the other.
2. Hits yes, no, yes, no with 2 truths: $(TP, FP) = (1,0), (1,1), (2,1), (2,2)$; precision $1, 0.5, 0.67, 0.5$; recall $0.5, 0.5, 1, 1$. Envelope: 1 up to recall 0.5, then 0.67 up to 1: $\text{AP} = 0.5 \cdot 1 + 0.5 \cdot 0.67 = 0.833$. With yes, yes, no, no: precision $1, 1, 0.67, 0.5$, recall $0.5, 1, 1, 1$: $\text{AP} = 1.0$; the two false positives come after full recall and cost nothing. Order by score is everything.
3. Three pools need a size divisible by 8: $100 \to 50 \to 25 \to 12$ (floor), up $12 \to 24$ against the skip of 25: it fails at the first `cat`. Down: $100, 50, 25, 12$; up would be $24, 48, 96$. Resize (or pad) the slices to 96 or 104 in the Dataset, a multiple of 8, and resize the predicted masks back with `nearest`.
4. The teammate's. AP is the area under the whole precision-recall curve; a file cut at `conf=0.9` contains only the first part of the curve (recall stops at 0.4, the rest of the area is zero), while the `conf=0.01` file contains all the boxes and lets the metric sweep every threshold; the extra low-score false positives come after the hits and do not lower the envelope. Same model, different fraction of its curve handed in.

Metrics: **Limitations & What Comes Next**

Limitations

- **The YOLO arithmetic is the 2016 paper's.** The ultralytics loss uses CIoU for boxes, binary cross-entropy for classes and a distribution over box-side distances; the three jobs are the same, the formulas are not, and the deck quotes them only by their `results.csv` names.
- **The AP conventions differ in the details.** All-point (VOC 2010), 11-point (VOC 2007) and 101-point (COCO, ultralytics) give 0.854, 0.864 and 0.856 on the table; the rules name one. Matching is greedy by score, each truth used once; COCO's crowd regions are not covered here.
- **Mask metrics have the same fine print:** Dice per image or per dataset, mean IoU with or without background, mask mAP for instances. Compute what the rules compute, on your validation split.

The common mistake

Submitting the boxes you would *show*: a high `conf`, a tight NMS, one box per object. A mAP leaderboard wants the *whole* ranked list; keep the low-score boxes, lift the NMS threshold when objects touch, and let the metric cut the curve where it likes.

What Comes Next

- **Day 27, Olympiad practice:** the vision habits for Friday: augmentation that moves boxes and masks with the photo, test-time augmentation, the checklist of Day 25 and the metric reading of this deck as a routine.
- **Friday:** the Week 4 competition.
- **Week 5:** the pretrained backbones inside YOLO and every modern U-Net, explained: transfer learning, fine-tuning, embeddings.

Metrics: **Summary**

Summary

- **A YOLO cell** decodes to $c_x = (\text{col} + x)\,\text{cell}_w$, $W = w \cdot \text{width}$, $\text{score} = C \cdot p(\text{class})$; the 2016 loss weights box terms by 5 and empty cells by 0.5: 0.190 on our cell.
- **NMS by hand:** keep the best, delete overlaps above the threshold, repeat; touching objects need a higher `iou` at `predict`.
- **The modern head:** 8,400 positions at 640 pixels, no anchors; `results.csv` holds three losses and four metrics per epoch; `best.pt` is chosen by mAP50-95.
- **The small U-Net:** 467,809 parameters, half in `bottom`; `cat` doubles the channels; the input must divide by 2^{pools}. Mask loss: BCE with `pos_weight` plus Dice loss; threshold at logit 0; resize back with `nearest`.
- **On the grid:** *TP* 4, *FP* 5, *FN* 5: IoU 0.286, Dice 0.444, accuracy 0.722 against 0.750 for all-background; $\text{Dice} = 2\,\text{IoU}/(1 + \text{IoU})$.
- **AP by hand:** sort by score, cumulative *TP* and *FP*, the envelope, its area: 0.854 at IoU 0.5, 0.375 at 0.75 for the same boxes; mAP averages over classes. Submit the whole ranked list.

Metrics: **References & Resources**

References & Resources

[1] Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016). You only look once: unified, real-time object detection. *CVPR*. Section 2: the cell parameterisation, the confidence as IoU, the loss with $\lambda_{\text{coord}} = 5$ and $\lambda_{\text{noobj}} = 0.5$.

[2] Ultralytics YOLO documentation: the YOLOv8 and YOLO11 anchor-free head with strides 8, 16 and 32; `train` outputs and `results.csv`; `val` and `predict` arguments (`conf`, `iou`, `imgsz`); the `Results.boxes` attributes; the metrics object and the fitness definition. https://docs.ultralytics.com

[3] Ronneberger, O., Fischer, P. and Brox, T. (2015). U-Net: convolutional networks for biomedical image segmentation. *MICCAI*. Fig. 1: 572 $\rightarrow$ 388 with unpadded convolutions and cropped skips.

[4] Milletari, F., Navab, N. and Ahmadi, S.-A. (2016). V-Net: fully convolutional neural networks for volumetric medical image segmentation. *3DV*. The Dice loss.

[5] Everingham, M., Van Gool, L., Williams, C., Winn, J. and Zisserman, A. (2010). The PASCAL visual object classes (VOC) challenge. *IJCV* 88. Matching at IoU 0.5, each truth once; the 11-point and the all-point average precision.

[6] Lin, T.-Y. et al. (2014). Microsoft COCO: common objects in context. *ECCV*. mAP over IoU 0.5 to 0.95, the 101-point interpolation.

[7] torchvision documentation: `ops.box_iou`, `ops.nms`, `nn.functional.interpolate` (`mode="nearest"` for masks). https://pytorch.org/vision/stable/ops.html

References & Resources (cont.)

[8] PyTorch documentation: `nn.BCEWithLogitsLoss` (`pos_weight`), `nn.CrossEntropyLoss` for k-class masks. https://pytorch.org/docs/stable/nn.html

[9] Wang, L., Shi, J., Song, G. and Shen, I. (2007). Object detection combining recognition and segmentation. *ACCV*. The Penn-Fudan photo of the grid frame. https://www.cis.upenn.edu/~jshi/ped_html/

[10] Kaggle datasets of the practice task and of Day 24's labs: `andrewmvd/road-sign-detection`, `nikhilroxtomar/brain-tumor-segmentation`.

[11] Figures and numbers without another source: `slides-builder/scripts/day26_metrics_figures.py` (NMS, the precision-recall table and curves, the mask grid, Dice against IoU, the YOLO-v1 loss terms, the U-Net shapes and parameter counts, the BCE and Dice values), all computed with torch and torchvision; no model was trained for this deck.